MIP heuristics

Outline

- MIP-based heuristics
- The examples of the chapter
- Rounding
- Diving
- The feasibility pump
- Improving an incumbent with a sub-MIP
  - RINS
  - Local branching
  - Proximity search
- Relax-and-fix and fix-and-optimize
  - Relax-and-fix
  - Fix-and-optimize
- Heuristics inside the solver
- What MIP-based heuristics give
- `Python` implementation
- Typical mistakes
- In summary

Learning objectives

At the end of this chapter you will be able to:

- explain how a heuristic algorithm uses the LP relaxation and a MIP solver;
- execute by hand rounding, diving, the feasibility pump, RINS, local branching and proximity search on a small MIP;
- execute relax-and-fix and fix-and-optimize on a MIP whose variables are split into blocks;
- decide which method to use when a feasible solution is missing, or when an incumbent solution must be improved;
- compute the gap of a heuristic solution when a dual bound is available.

Prerequisites

- From this module, the chapters «Primal and dual bounds» and «LP-based branch-and-bound algorithms»: the LP relaxation, the lower and upper bounds, the optimality gap, and the branching operations
- From this module, the chapter «MIP solvers: what they do»: the incumbent solution, the sub-problems solved by a solver, and the heuristics inside it
- From the module *Linear Programming*: the dual simplex algorithm (Chapter 10), which re-optimizes an LP after a change of a bound

MIP-based heuristics

Introduction: MIP-based heuristics

- Branch-and-bound finds good solutions only by chance, as leaves of the tree: often a good solution is needed **early**, and quickly
- A MIP-based heuristic uses two tools of this module: the **LP relaxation**, to decide where to look, and the **MIP solver**, to explore exactly a large set of solutions
- We fix the notation of the chapter: the MIP in general form, the incumbent, the rounding and the completion of a point
- We meet the five ideas behind all the methods of the chapter

Definition 1: MIP in general form (1/2)

*A **MIP** with N variables $\boldsymbol{x} = (x_1, x_2, \ldots, x_N)$ reads*

$$\begin{aligned} \min \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & A\boldsymbol{x} \leq b, \\ & l_j \leq x_j \leq u_j, \quad \forall j \in \{1, 2, \ldots, N\}, \\ & x_j \in \mathbb{Z}, \qquad\qquad \forall j \in \mathscr{D}, \end{aligned}$$

*where f is a linear function and $\mathscr{D} \subseteq \{1, 2, \ldots, N\}$ is the set of the indices of the **integer variables**.*

Definition: MIP in general form (2/2)

The feasible region of the ***LP relaxation*** *is*

$$\mathcal{L} = \left\{ \boldsymbol{x} \in \mathbb{R}^N : A\,\boldsymbol{x} \leq b,\ l \leq \boldsymbol{x} \leq u \right\},$$

and

$$z(\mathrm{LP}) = \min\{ f(\boldsymbol{x}) : \boldsymbol{x} \in \mathcal{L} \}$$

is the value of the LP relaxation.

- A maximization problem, such as the car example, is handled in the same way, exchanging min and max, $\leq$ and $\geq$ in the comparisons of objective values

Definition 2: MIP-based heuristic

*A **heuristic algorithm** returns a feasible solution of a MIP problem without any guarantee of optimality. It is **MIP-based** (a **matheuristic**) if it uses the LP relaxation and a MIP solver to search for the solution.*

- A heuristic gives a primal bound (an upper bound for a minimization problem, a lower bound for a maximization problem), never a dual bound
- The branch-and-bound algorithm of this module finds the incumbent solution only by chance, as a leaf of the tree

The two tools of a MIP-based heuristic

- The optimal solution of the **LP relaxation** tells *where to look*: it is the solution of a simpler problem, often close to the good solutions of the original one

- The **MIP solver** explores *exactly* a set of solutions far too large to be enumerated, provided that the set is described by linear constraints

Definition 3: incumbent, rounding and completion

- *A feasible solution $\bar{\boldsymbol{x}}$ of the MIP, the best one known, is the **incumbent**, and $f(\bar{\boldsymbol{x}})$ is its value.*
- *The **rounding** of a real number v to the nearest integer is $[\,v\,] = \lfloor v + \frac{1}{2} \rfloor$, so that a value $\frac{1}{2}$ is rounded up.*
- *A point $\bar{\boldsymbol{x}}$ with $\bar{x}_j \in \mathbb{Z}$ for all $j \in \mathscr{D}$ is **completed** by fixing the integer variables to the values $\bar{x}_j$ and solving the LP on the continuous variables.*

Notation of the chapter

- Throughout the chapter, $\tilde{\boldsymbol{x}}$ denotes an optimal solution of an LP relaxation, and $\bar{\boldsymbol{x}}$ a point whose components $\bar{x}_j$, $j \in \mathscr{D}$, are integer
- If the LP of the completion has an optimal solution, $\bar{\boldsymbol{x}}$ is turned into a feasible solution of the MIP; if it is infeasible, so is $\bar{\boldsymbol{x}}$
- When all the variables are integer, the completion simply checks the constraints

Five ideas

- The methods are built on five ideas, one or two per section:

idea	methods
round the relaxation	rounding; diving (fix one variable, re-solve, repeat)
chase feasibility	the feasibility pump
fix where relaxation and incumbent agree	RINS
bound the distance from the incumbent	local branching; proximity search
decompose into blocks of variables	relax-and-fix; fix-and-optimize

The solver of the examples

- The examples are solved with the MIP solver Gurobi, used from Python through `gurobipy`
- The module *MIP Modelling* (fabiofurini.github.io/mip-modelling, chapter 3 «From the model to Python/Gurobi») explains how to use it
- When an LP has several optimal solutions, we report the one returned by the solver
- All the codes are in the section on the Python implementation

Summary

- A **MIP-based heuristic** (matheuristic) finds a feasible solution with the LP relaxation and a MIP solver, without any guarantee of optimality: it gives a primal bound only
- $\tilde{\boldsymbol{x}}$ is an optimal LP solution, $\bar{\boldsymbol{x}}$ a point integer on $\mathscr{D}$; the **incumbent** is the best feasible solution known
- $[\,v\,] = \lfloor v + \frac{1}{2} \rfloor$ is the rounding; the **completion** fixes the integer variables and solves the LP on the continuous ones
- Five ideas: round, chase feasibility, fix where LP and incumbent agree, bound the distance, decompose into blocks

The examples of the chapter

Introduction: The examples of the chapter

- All the methods of the chapter run on the same small problems, so that they can be compared
- We meet again the car example, and four small problems of the catalogue of MIP problems
- For each problem we know the LP value, the upper bound and the optimal value: we can always measure how good a heuristic solution is

The car example and the catalogue

- The methods are illustrated on the **car example** of this module and on four small problems of the **catalogue of MIP problems**
- All of them are ILP problems in canonical maximization form, with integer data and $x_j \in \mathbb{Z}_{\geq 0}$ for every j, so that $\mathscr{D} = \{1, 2, \ldots, N\}$
- The **car example** reads:

$$\begin{array}{rrcrcr} \max & 3\,x_1 & + & 2\,x_2 & & \\ \text{subject to} & 10\,x_1 & + & 4\,x_2 & \leq & 45 \\ & 4\,x_1 & + & 14\,x_2 & \leq & 49 \\ & x_1, & & x_2 & \in & \mathbb{Z}_{\geq 0} \end{array}$$

The problem *P7*

$$
\begin{aligned}
\max \quad & 3x_1 + x_2 \\
\text{subject to} \quad & x_1 + 4x_2 \leq 12, \\
& 2x_1 + x_2 \leq 5, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2\}
\end{aligned}
$$

The problem $P42$

$$
\begin{aligned}
\max \quad & 3x_1 + 2x_2 + 4x_3 \\
\text{subject to} \quad & x_1 + x_2 + 2x_3 \leq 4, \\
& 2x_1 + 3x_3 \leq 5, \\
& 2x_1 + x_2 + 3x_3 \leq 7, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, 3\}
\end{aligned}
$$

The problem $P46$

$$
\begin{aligned}
\max \quad & 5x_1 + 6x_2 + 10x_3 \\
\text{subject to} \quad & 9x_1 + 2x_2 + 2x_3 \leq 11, \\
& 2x_1 + 6x_2 + 6x_3 \leq 16, \\
& 4x_1 + 2x_2 + 7x_3 \leq 10, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, 3\}
\end{aligned}
$$

The problem *P72*

$$
\begin{aligned}
\max \quad & 7x_1 + 6x_2 + 5x_3 - 2x_4 + 3x_5 \\
\text{subject to} \quad & x_1 + 3x_2 + 5x_3 - 2x_4 + 2x_5 \leq 4, \\
& 4x_1 + 2x_2 - 2x_3 + x_4 + x_5 \leq 3, \\
& 2x_1 + 4x_2 + 4x_3 - 2x_4 + 5x_5 \leq 5, \\
& 3x_1 + x_2 + 2x_3 - x_4 - 2x_5 \leq 1, \\
& x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \ldots, 5\}
\end{aligned}
$$

Bounds and optimal values

- The table reports the value $z(\mathrm{LP})$ of the LP relaxation, the upper bound $\lfloor z(\mathrm{LP}) \rfloor$ (the objective takes integer values), and the optimal value $z(\mathrm{ILP})$
- All computed with the exact simplex algorithm and checked with Gurobi

problem	variables	constraints	$z(\mathrm{LP})$	$\lfloor z(\mathrm{LP}) \rfloor$	$z(\mathrm{ILP})$
car	2	2	$\frac{31}{2}$	15	14
$P7$	2	2	$\frac{15}{2}$	7	7
$P42$	3	3	$\frac{21}{2}$	10	10
$P46$	3	3	$\frac{499}{25}$	19	16
$P72$	5	4	$\frac{507}{59}$	8	7

Example: the LP solutions of the examples

problem	optimal solution $\tilde{\boldsymbol{x}}$ of the LP relaxation
car	$\left(\frac{7}{2}, \frac{5}{2}\right)$
$P7$	$\left(\frac{5}{2}, 0\right)$
$P42$	$\left(\frac{5}{2}, \frac{3}{2}, 0\right)$
$P46$	$\left(\frac{17}{25}, \frac{49}{25}, \frac{12}{25}\right)$
$P72$	$\left(\frac{39}{59}, 0, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}\right)$

- The first three solutions have at most two fractional components, with fractional part $\frac{1}{2}$
- The last two are the examples where the methods have more room to fail

Summary

- Five small ILP problems in canonical maximization form: the **car example**, *P*7, *P*42, *P*46 and *P*72, all with $\mathscr{D} = \{1, 2, \ldots, N\}$
- For each one we know $z(\mathrm{LP})$, the upper bound $\lfloor z(\mathrm{LP}) \rfloor$ and $z(\mathrm{ILP})$
- The bound is tight on *P*7 and *P*42, not on the car example, *P*46 and *P*72
- *P*46 and *P*72 have LP solutions with many fractional components: there the methods have more room to fail

Rounding

Introduction: Rounding

- The simplest idea: solve the LP relaxation and round its solution
- We see when rounding works, when it fails, and a case in which rounding *down* is always safe
- On the examples, rounding to the nearest integer fails three times: this motivates the methods of the next sections

Definition 4: rounding

Rounding is the simplest MIP-based heuristic. It solves the LP relaxation, rounds every integer variable of the LP solution to the nearest integer, and completes the continuous variables. It returns a feasible solution, or it fails if the rounded point violates a constraint.

- The pseudo-code of rounding is on the next two slides

Algorithm: Rounding (1/2)

Input : MIP with integer variables $\mathscr{D}$
Output : Feasible solution $\bar{\boldsymbol{x}}$, or FAIL

```
1 function Rounding(MIP)
2 |   x̃ := optimal solution of the LP relaxation
  |    // min{f(x) : x ∈ L}
3 |   for j ∈ D do
4 |   |   x̄_j ← [x̃_j]        // round to the nearest integer
5 |   end
6 |   fix x_j := x̄_j for all j ∈ D and solve the LP on the
  |    continuous variables
```

Algorithm: Rounding (2/2)

function *Rounding(MIP)* (continued)

```
7   |  if the LP is feasible then
8   |  |  return x̄ completed with the continuous part of the
    |  |    LP solution
9   |  else
10  |  |  return FAIL   // the rounded point violates a
    |  |    constraint
11  |  end
12  end
```

Why rounding can work

- When the formulation is **strong**, the LP solution is close to an optimal integer solution
- Many of its integer variables are already integer
- Rounding changes only a few of them, by less than $\frac{1}{2}$ each

Why rounding can fail

- Rounding treats the variables one at a time, while the constraints link them
- A constraint with all non-negative coefficients and right-hand side $b_i \geq 0$ stays satisfied if the variables are rounded *down*, and one of the form $\geq$ if they are rounded *up*
- An **equality** constraint is almost always violated by independent roundings: two fractional values that sum to 1 become, for instance, 1 and 1

- When all the data are non-negative, rounding down cannot violate a constraint:

Observation 1

Let all the coefficients of A be non-negative, $b \geq 0$ and $l = 0$. If $\tilde{\boldsymbol{x}} \in \mathcal{L}$, then the point $\lfloor \tilde{\boldsymbol{x}} \rfloor$, obtained by rounding down every integer variable, belongs to $\mathcal{L}$.

Proof

- **Bounds.** For every j:

$$0 \leq \lfloor \tilde{x}_j \rfloor \leq \tilde{x}_j \leq u_j,$$

  so the bounds hold

- **Constraints.** For every row i, since $a_{ij} \geq 0$:

$$\sum_j a_{ij} \lfloor \tilde{x}_j \rfloor \leq \sum_j a_{ij} \tilde{x}_j \leq b_i$$



The car example satisfies the assumptions of the observation:
rounding down is always feasible,
and in general not optimal.

Let us see what happens on the examples.

Example (1/3): rounding on the car example

- The LP solution $\tilde{\boldsymbol{x}} = \left(\frac{7}{2}, \frac{5}{2}\right)$ is rounded to $\bar{\boldsymbol{x}} = (4, 3)$, which violates the first constraint:

$$10 \cdot 4 + 4 \cdot 3 = 52 > 45$$

- **Rounding fails**

- Rounding down instead gives $(3, 2)$:

$$10 \cdot 3 + 4 \cdot 2 = 38 \leq 45, \qquad 4 \cdot 3 + 14 \cdot 2 = 40 \leq 49,$$

  of value 13, while $z(\mathrm{ILP}) = 14$

Example (2/3): rounding on $P42$

- The LP solution $\left(\frac{5}{2}, \frac{3}{2}, 0\right)$ is rounded to $(3, 2, 0)$, which violates the second constraint:

$$2 \cdot 3 = 6 > 5$$

- **Rounding fails**
- Rounding down gives $(2, 1, 0)$, of value 8, while $z(\mathrm{ILP}) = 10$

Example (3/3): rounding on $P72$

- The LP solution $\left(\frac{39}{59}, 0, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}\right)$ is rounded to $(1, 0, 2, 3, 1)$, which violates the first constraint:

$$1 + 10 - 6 + 2 = 7 > 4$$

- **Rounding fails**

Summary

- **Rounding** solves the LP relaxation, rounds the integer variables to the nearest integer and completes the continuous ones
- It fails when several variables are fractional at the same time and the constraints are tight: car example, $P42$, $P72$
- With non-negative data, **rounding down** is always feasible, but it leaves part of the LP value on the table: 13 against 14 on the car example, 8 against 10 on $P42$

Diving

Introduction: Diving

- Rounding takes all its decisions at once, on the basis of a single LP solution
- **Diving** takes them one at a time, and after each decision it asks the LP again: the LP solution is re-computed with the decisions taken so far, and it adapts to them
- We see two rules (fractional and guided diving), the link with branch-and-bound, and dives that succeed and dives that fail

Definition 5: diving

Diving repeatedly solves the LP relaxation, selects one integer variable with a fractional value, fixes it to an integer value, and solves the LP again. It stops with a feasible solution when the LP solution is integer on all the integer variables, and it fails when the LP becomes infeasible. A rule decides which variable is fixed and to which value.

- The pseudo-code of diving is on the next two slides

Algorithm: Diving (1/2)

Input : MIP; a rule; for guided diving, an incumbent $\bar{\boldsymbol{x}}$
Output : Feasible solution, or FAIL

```
1  function Diving(MIP, rule)
2      while true do
3          solve the LP relaxation with the current bounds l, u
4          if the LP is infeasible then
5              return FAIL          // the dive ends in an
                infeasible LP
6          end
7          x̃ := optimal solution of the LP
8          F := {j ∈ 𝒟 : x̃_j ∉ ℤ}      // fractional integer
            variables
```

Algorithm: Diving (2/2)

function *Diving(MIP, rule)* (continued)

    while *true* (continued) **do**
        if $F = \emptyset$ **then**
            return $\tilde{x}$ `// integer on` $\mathscr{D}$`: feasible`
        end
        select $j \in F$ and a value $v \in \mathbb{Z}$ according to the rule
        $l_j \leftarrow v$; $u_j \leftarrow v$ `// fix` $x_j := v$
    end
end

Two rules

- **Fractional diving.** Select the variable that is *least* fractional, i.e., the $j \in F$ that minimizes $\left|\tilde{x}_j - [\,\tilde{x}_j\,]\right|$, and fix it to $v = [\,\tilde{x}_j\,]$
  - The LP is almost sure about this variable, so the fixing changes the LP solution as little as possible
  - In the codes of this chapter, ties are broken in favour of the variable with the smallest index

- **Guided diving** (Danna, Rothberg and Le Pape, 2005). With an incumbent $\bar{\boldsymbol{x}}$, select the $j \in F$ that minimizes $|\tilde{x}_j - \bar{x}_j|$, and fix it to $v = \bar{x}_j$
  - The incumbent guides the dive toward a region of good solutions, and the LP decides in which order its values are adopted

Other rules, and the link with branch-and-bound

- Many other rules exist: *coefficient diving* fixes the variable that appears in the fewest constraints that can be violated by rounding it; *pseudo-cost diving* uses the history of the branch-and-bound tree; *vector-length diving* is tailored to set covering constraints. Modern solvers run several of them

- **Diving and branch-and-bound.** Fixing $x_j := [\tilde{x}_j]$ is a **branching decision**

- A dive is a single path from the root of the branch-and-bound tree to a leaf, always taking one of the two children, without ever coming back: a **depth-first search without backtracking**

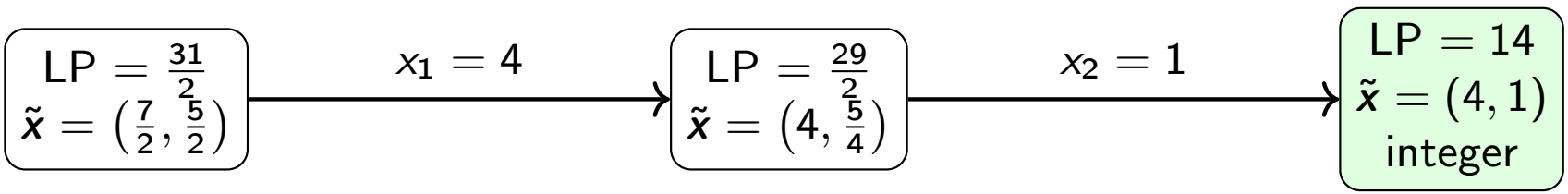
Cost and failure

- **Cost.** Every step solves an LP that differs from the previous one by a single bound, so the dual simplex algorithm re-optimizes it starting from the previous basis in a few iterations
- A dive with $|\mathscr{D}|$ integer variables needs at most $|\mathscr{D}|$ LPs, usually far fewer, since an LP solution has many integer components already
- **Failure.** The LP becomes infeasible when the fixings taken so far are incompatible
- Implementations then undo the last fixing and try the other value (a single backtrack), or give up

Example (1/2): fractional diving on the car example

- **Step 1.** The LP solution is $\tilde{\boldsymbol{x}} = \left(\frac{7}{2}, \frac{5}{2}\right)$, of value $\frac{31}{2}$. Both variables are at distance $\frac{1}{2}$ from the nearest integer; the tie is broken by the index: $x_1 := \left[\frac{7}{2}\right] = 4$
- **Step 2.** With $x_1 = 4$ the constraints become $4x_2 \leq 5$ and $14x_2 \leq 33$, so the LP solution is $\tilde{\boldsymbol{x}} = \left(4, \frac{5}{4}\right)$, of value $12 + \frac{5}{2} = \frac{29}{2}$. Fix $x_2 := \left[\frac{5}{4}\right] = 1$
- **Step 3.** The LP solution $(4, 1)$, of value 14, is integer: the dive returns $(4, 1)$, which is the **optimal** solution of the ILP problem

Example (2/2): fractional diving on the car example



- The dive is a single path of the branch-and-bound tree: two fixings, three LPs

Example (1/2): guided diving on the car example

- The incumbent is $\bar{\boldsymbol{x}} = (3, 2)$, of value 13 (rounding down)
- In the LP solution $\left(\frac{7}{2}, \frac{5}{2}\right)$ both variables are at distance $\frac{1}{2}$ from $\bar{\boldsymbol{x}}$; fix $x_1 := 3$
- The new LP solution is $\left(3, \frac{37}{14}\right)$, of value $\frac{100}{7}$; fix $x_2 := \bar{x}_2 = 2$
- The LP solution $(3, 2)$ is integer, of value 13: the dive **returns the incumbent**, without improving it

Example (2/2): guided diving on $P7$

- The incumbent is $\bar{\boldsymbol{x}} = (2, 0)$, of value 6 (rounding down of $\left(\frac{5}{2}, 0\right)$)
- The only fractional variable is x_1: fix $x_1 := 2$
- The LP solution $(2, 1)$, of value 7, is integer: guided diving finds the **optimal** solution after one fixing
- This happens even if the incumbent has $\bar{x}_2 = 0$, because the LP rebuilds the variables that are not fixed

The incumbent only decides the *values*
of the fixed variables:

on the car example both fixed values
agree with the poor incumbent,
on $P7$ the fixed value is the right one.

Example (1/2): a dive that fails on $P42$

- The LP solution is $\left(\frac{5}{2}, \frac{3}{2}, 0\right)$; x_1 and x_2 are at distance $\frac{1}{2}$, and the tie is broken by the index: $x_1 := \left[\frac{5}{2}\right] = 3$
- The second constraint gives

$$2 \cdot 3 + 3x_3 \leq 5,$$

  which is violated for every $x_3 \geq 0$
- The LP is infeasible: **the dive fails at the first step**

Example (2/2): a dive that fails on $P72$

- The fractional components of the LP solution are $\frac{39}{59}, \frac{91}{59}, \frac{166}{59}, \frac{37}{59}$ (for x_1, x_3, x_4, x_5)
- The least fractional is x_4, at distance $\frac{11}{59}$ from 3: $x_4 := 3$
- The new LP has value $\frac{728}{85}$ and solution $(\frac{56}{85}, 0, \frac{138}{85}, 3, \frac{52}{85})$
- The least fractional variable is x_1, rounded to 1
- Fixing $x_1 := 1$ and $x_4 := 3$ makes the LP **infeasible**

- In both cases the first fixings follow the LP, which does not see that the constraints, together, leave no room
- A dive that fails gives no solution at all
- This is the reason for the feasibility pump of the next section

Summary

- **Diving** fixes one fractional variable at a time and re-solves the LP: a single path of the branch-and-bound tree, without backtracking
- **Fractional diving** fixes the least fractional variable to its rounding; **guided diving** fixes the variable closest to the incumbent to the incumbent value
- Each step is cheap: the dual simplex algorithm re-optimizes from the previous basis
- On the car example fractional diving finds the optimum $(4, 1)$; on $P42$ and $P72$ the dive ends in an infeasible LP

The feasibility pump

Introduction: The feasibility pump

- Rounding and diving fail on $P42$ and on $P72$, and on many MIPs they fail most of the time: for a general MIP, even *deciding whether a feasible solution exists* is NP-complete
- The **feasibility pump** of Fischetti, Glover and Lodi (2005) gives up the objective function and concentrates on a single goal: to find a **feasible** solution
- We define the distance from an integer point, the pump with its perturbations, and we run it on three examples

Two points that chase each other

- The idea is to work with two points at the same time:
  - a point of $\mathcal{L}$ satisfies all the linear constraints, but may be fractional
  - an integer point is integer, but may violate the constraints
- A feasible solution is a point that is both
- The pump makes the two points chase each other until they coincide, and to do so it needs to measure how far apart they are

Definition 6: Distance from an integer point

*Let $\bar{\boldsymbol{x}}$ be a point with $\bar{x}_j \in \mathbb{Z}$ for all $j \in \mathscr{D}$. The **distance** of a point $\boldsymbol{x} \in \mathbb{R}^N$ from $\bar{\boldsymbol{x}}$ is*

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}} |x_j - \bar{x}_j|,$$

the L_1 distance computed on the integer variables only.

The distance for binary variables

- When all the integer variables are **binary**, $|x_j - \bar{x}_j|$ equals x_j if $\bar{x}_j = 0$ and $1 - x_j$ if $\bar{x}_j = 1$, for every $x_j \in [0, 1]$
- The distance is then a *linear* function of $\boldsymbol{x}$:

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) = \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j + \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j)$$

for binary variables

The distance for general integer variables

- For a general integer variable the same holds if $\bar{x}_j = l_j$ (the term is $x_j - l_j$) or $\bar{x}_j = u_j$ (the term is $u_j - x_j$)
- Otherwise $|x_j - \bar{x}_j|$ is replaced by an auxiliary variable d_j with the two constraints

$$d_j \geq x_j - \bar{x}_j, \qquad d_j \geq \bar{x}_j - x_j;$$

  at the minimum, $d_j = |x_j - \bar{x}_j|$
- In all cases, the point of $\mathcal{L}$ closest to $\bar{\boldsymbol{x}}$ is computed by solving **one LP**

Definition 7: feasibility pump (1/2)

The feasibility pump builds two sequences of points:

- *points* $\tilde{\boldsymbol{x}}^0, \tilde{\boldsymbol{x}}^1, \tilde{\boldsymbol{x}}^2, \ldots$ *of* $\mathcal{L}$*, which satisfy the linear constraints;*
- *integer points* $\bar{\boldsymbol{x}}^0, \bar{\boldsymbol{x}}^1, \bar{\boldsymbol{x}}^2, \ldots$*, which satisfy the integrality constraints.*

It starts from the LP solution $\tilde{\boldsymbol{x}}^0$*.*

Definition: feasibility pump (2/2)

Each integer point is the rounding of the last point of $\mathcal{L}$, and each point of $\mathcal{L}$ is the projection of the last integer point, i.e., the point of $\mathcal{L}$ at minimum distance from it. It stops when the distance is 0: the two points coincide and are a feasible solution. When the rounding returns an integer point already visited, the pump perturbs it.

- The pseudo-code of the feasibility pump is on the next three slides

Algorithm: Feasibility pump (1/3)

Input : MIP; maximum number of iterations *maxIter*; seed of the random numbers

Output : Feasible solution, or FAIL

```
1  function FeasibilityPump(MIP, maxIter, seed)
2      x̃ := optimal solution of the LP relaxation
3      if x̃_j ∈ ℤ for all j ∈ 𝒟 then
4          return x̃
5      end
6      x̄ ← [x̃]                    // round the integer components
7      for it := 1 to maxIter do
8          x̃ ← arg min{Δ(x, x̄) : x ∈ ℒ}      // projection: one LP
9          if Δ(x̃, x̄) = 0 then
10             return x̃  // x̃ is integer on 𝒟:  feasible
11         end
12         x̂ ← [x̃]
```

Algorithm: Feasibility pump (2/3)

function *FeasibilityPump(MIP, maxIter, seed)* (continued)

```
    for it := 1 to maxIter (continued) do
13      if x̂_j = x̄_j for all j ∈ 𝒟 then                    // short cycle
14          j ← arg max_{k∈𝒟} |x̃_k − x̄_k|
15          move x̂_j by one unit toward x̃_j                  // flip
16      end
17      if x̂ equals one of the last 3 points x̄ then        // long
          cycle
18          for j ∈ 𝒟 do
19              ρ_j ← uniform random number in [−0.3, 0.7]
20              if |x̃_j − x̂_j| + max{ρ_j, 0} > 1/2 then
21                  x̂_j ← 1 − x̂_j                          // restart
22              end
23          end
24      end
```

Algorithm: Feasibility pump (3/3)

function *FeasibilityPump(MIP, maxIter, seed)* (continued)

    for *it* := 1 **to** *maxIter* (continued) **do**

25      $\bar{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}}$

26 **end**

27 **return** *FAIL*

28 **end**

Why the pump works

- Each **projection** moves the point of $\mathcal{L}$ toward the last integer point
- Each **rounding** moves the integer point to the nearest one
- The distance between the two sequences tends to decrease, and the points are *pumped* toward a region where $\mathcal{L}$ contains integer points
- Each iteration costs one LP and a rounding

Short cycles

- If the rounding of the projection gives back the same integer point, the pump would repeat the same two points forever
- It then changes the components of $\bar{\boldsymbol{x}}$ on which $\tilde{\boldsymbol{x}}$ is most uncertain, those with the largest $|\tilde{x}_j - \bar{x}_j|$
- The original pump flips a random number of components between 10 and 30
- On the small instances of these slides one flip is enough: the code flips the single component with the largest $|\tilde{x}_j - \bar{x}_j|$ (the first one, in case of ties)

Long cycles and random numbers

- **Long cycles.** The pump can also cycle among several integer points. When the new integer point equals one of the last ones, the original pump makes a **restart**:
  - a binary component is flipped when $|\tilde{x}_j - \hat{x}_j| + \max\{\rho_j, 0\} > \frac{1}{2}$, with ρ_j uniformly random in $[-0.3, 0.7]$
  - the components where $\tilde{x}_j$ is far from $\hat{x}_j$ are flipped with high probability

- **Random numbers.** In the codes of this chapter, they are produced by the *minimal standard* generator of Park and Miller, with integer arithmetic on 64 bits:

$$seed \leftarrow (16807 \cdot seed) \bmod (2^{31} - 1), \qquad u = \frac{seed}{2^{31} - 1} \in (0, 1),$$

and in the pump $\rho_j = u - 0.3$. Fixing the initial seed makes every run reproducible

The objective and general integer variables

- **The objective.** After the first LP, the original objective f is ignored: the pump finds a feasible solution, but not necessarily a good one
- The *objective feasibility pump* of Achterberg and Berthold (2007) minimizes a combination

$$(1-\alpha)\,\Delta(\boldsymbol{x},\bar{\boldsymbol{x}})+\alpha\,f(\boldsymbol{x})$$

  with a weight α that decreases at every iteration: the solutions found are much better, at the price of a few more iterations

- **General integer variables.** The pump was designed for binary variables, where the flip changes $\bar{x}_j$ into $1-\bar{x}_j$; for general integer variables it moves $\bar{x}_j$ by one unit toward $\tilde{x}_j$, and the distance uses the auxiliary variables d_j (Bertacco, Fischetti and Lodi, 2007)

We now run the pump by hand:
first on a problem in two variables, where we can draw
the two sequences of points,
then on the car example and on $P72$.

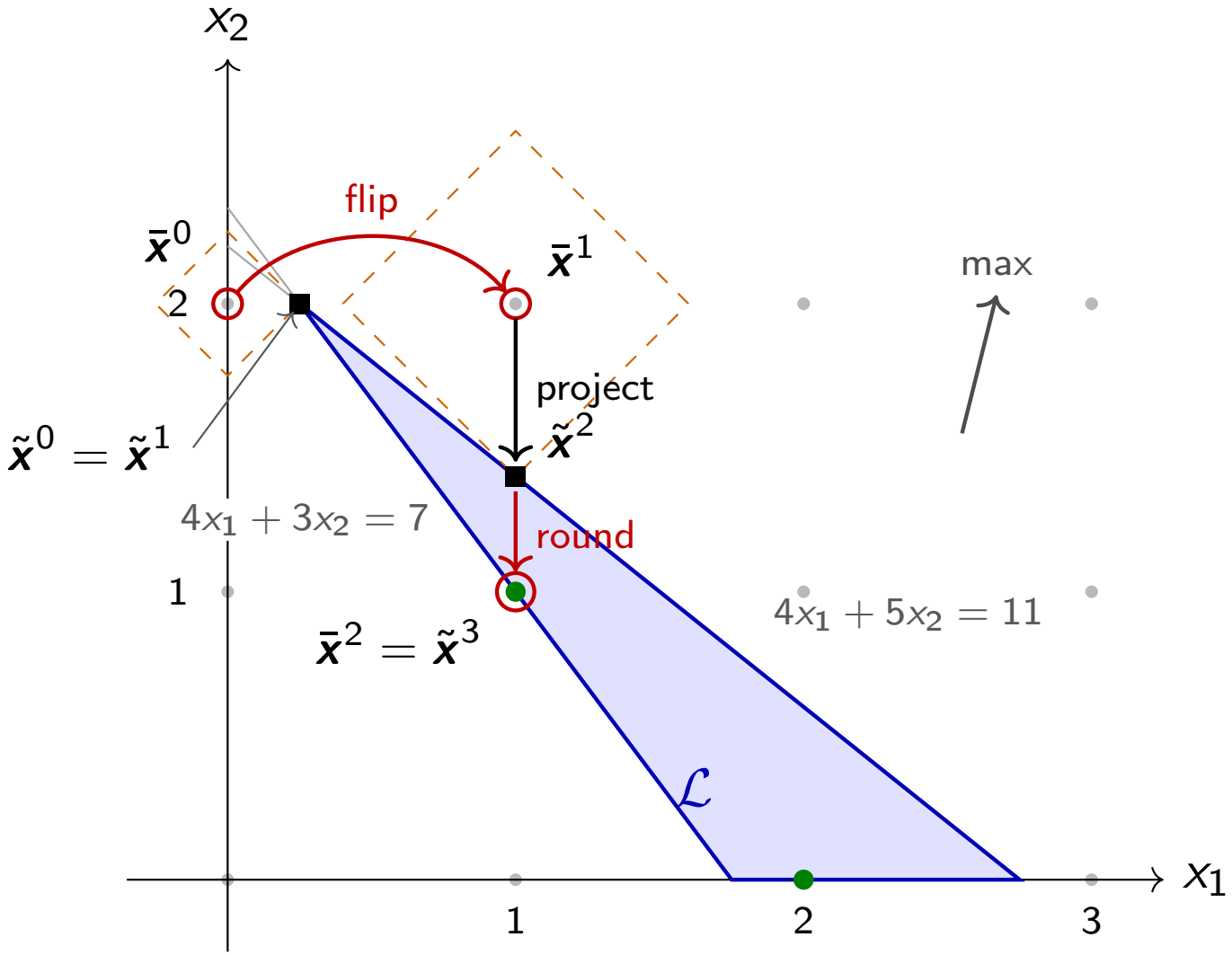
Example (1/7): the feasibility pump in two variables

- Consider the integer program

$$\begin{aligned} \max \quad & x_1 + 4\,x_2 \\ \text{subject to} \quad & 4\,x_1 + 3\,x_2 \geq 7 \\ & 4\,x_1 + 5\,x_2 \leq 11 \\ & x_1 \in \{0, 1, \ldots, 5\} \\ & x_2 \in \{0, 1, \ldots, 5\} \end{aligned}$$

- The region $\mathcal{L}$ is the triangle with vertices $\left(\frac{1}{4}, 2\right)$, $\left(\frac{7}{4}, 0\right)$, $\left(\frac{11}{4}, 0\right)$, which contains only two integer points, $(1, 1)$ and $(2, 0)$

Example (2/7): the feasibility pump in two variables



Squares: the points of $\mathcal{L}$; circles: the integer points; dashed: the diamonds $\{\boldsymbol{x} : \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq r\}$ of the two projections.

Example (3/7): the feasibility pump in two variables

- The iterations are the following:

k	integer point	projection $\tilde{\boldsymbol{x}}^k$	$\Delta(\tilde{\boldsymbol{x}}^k, \bar{\boldsymbol{x}}^{k-1})$	next integer point
0	—	LP optimum $(\frac{1}{4}, 2)$	—	rounding: $\bar{\boldsymbol{x}}^0 = (0, 2)$
1	$\bar{\boldsymbol{x}}^0 = (0, 2)$	$(\frac{1}{4}, 2)$	$\frac{1}{4}$	$[\tilde{\boldsymbol{x}}^1] = \bar{\boldsymbol{x}}^0$: flip x_1, $\bar{\boldsymbol{x}}^1 = (1, 2)$
2	$\bar{\boldsymbol{x}}^1 = (1, 2)$	$(1, \frac{7}{5})$	$\frac{3}{5}$	rounding: $\bar{\boldsymbol{x}}^2 = (1, 1)$
3	$\bar{\boldsymbol{x}}^2 = (1, 1)$	$(1, 1)$	0	stop: feasible

Example (4/7): the feasibility pump in two variables

- **Iteration 0.** The LP optimum is the vertex $\left(\frac{1}{4}, 2\right)$, of value $\frac{33}{4}$. Its rounding $(0, 2)$ violates the first constraint:

$$4 \cdot 0 + 3 \cdot 2 = 6 < 7$$

- **Iteration 1.** With $\bar{\boldsymbol{x}}^0 = (0, 2)$, x_1 is at its lower bound and

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^0) = x_1 + |x_2 - 2|$$

Example (5/7): the feasibility pump in two variables

- **Iteration 1 (continued).** The L_1 distance from a point to a segment that is neither horizontal, vertical nor of slope ± 1 is minimized at an endpoint or where the segment crosses the horizontal or the vertical line through the point
- So the candidates are the three vertices and these crossings
- The horizontal line $x_2 = 2$ touches $\mathcal{L}$ only at the vertex $\left(\frac{1}{4}, 2\right)$, and the vertical line $x_1 = 0$ misses it
- The closest point is $\tilde{\boldsymbol{x}}^1 = \left(\frac{1}{4}, 2\right)$, at distance $\frac{1}{4}$

Example (6/7): the feasibility pump in two variables

- **Iteration 1 (continued).** The rounding of $\tilde{\boldsymbol{x}}^1$ is $(0, 2) = \bar{\boldsymbol{x}}^0$ again: a **short cycle**
- The component with the largest $|\tilde{x}_j^1 - \bar{x}_j^0|$ is x_1 ($\frac{1}{4}$ against 0), which moves one unit toward $\frac{1}{4}$: $\bar{\boldsymbol{x}}^1 = (1, 2)$
- **Iteration 2.** The point $(1, 2)$ violates the second constraint $(14 > 11)$
- The vertical line $x_1 = 1$ crosses $\mathcal{L}$ between $(1, 1)$ and $\left(1, \frac{7}{5}\right)$, at distances 1 and $\frac{3}{5}$ from $(1, 2)$; the horizontal line $x_2 = 2$ gives $\left(\frac{1}{4}, 2\right)$, at distance $\frac{3}{4}$; the other two vertices are farther
- Hence $\tilde{\boldsymbol{x}}^2 = \left(1, \frac{7}{5}\right)$, and its rounding $(1, 1)$ is a new point

Example (7/7): the feasibility pump in two variables

- **Iteration 3.** The point $(1, 1)$ belongs to $\mathcal{L}$:

$$4 + 3 = 7 \geq 7, \qquad 4 + 5 = 9 \leq 11$$

- Its distance from $\mathcal{L}$ is 0, and the pump returns $(1, 1)$, of value 5

- It is **optimal**, since the other integer point $(2, 0)$ has value 2

Example (1/3): the feasibility pump on the car example

- Rounding the LP solution gives $\bar{\boldsymbol{x}}^0 = (4, 3)$, which is infeasible

k	projection $\tilde{\boldsymbol{x}}^k$	$\Delta(\tilde{\boldsymbol{x}}^k, \bar{\boldsymbol{x}}^{k-1})$	next integer point
1	$(\frac{7}{2}, \frac{5}{2})$	1	$[\tilde{\boldsymbol{x}}^1] = \bar{\boldsymbol{x}}^0$: flip x_1, $\bar{\boldsymbol{x}}^1 = (3, 3)$
2	$(3, \frac{37}{14})$	$\frac{5}{14}$	$[\tilde{\boldsymbol{x}}^2] = \bar{\boldsymbol{x}}^1$: flip x_2, $\bar{\boldsymbol{x}}^2 = (3, 2)$
3	$(3, 2)$	0	stop: feasible

Example (2/3): the feasibility pump on the car example

- At iteration 1 the two components of $\tilde{\boldsymbol{x}}^1$ are at the same distance $\frac{1}{2}$ from $\bar{\boldsymbol{x}}^0$, and the flip takes the first one
- The pump needs **two flips** and returns $(3, 2)$, of value 13
- It is feasible, but not optimal ($z(\mathrm{ILP}) = 14$), because after the first LP the objective is ignored

Example (3/3): the feasibility pump on *P*72

- Rounding and diving fail (see the examples of the previous sections)
- The pump rounds the LP solution to $\bar{\boldsymbol{x}}^0 = (1, 0, 2, 3, 1)$, which violates the first constraint
- It needs three projections and one flip (of x_3, from 2 to 1) to reach the feasible solution

$$\bar{\boldsymbol{x}} = (0, 0, 1, 3, 1), \qquad f(\bar{\boldsymbol{x}}) = 2,$$

very far from $z(\mathrm{ILP}) = 7$ and from the upper bound 8

The pump is a method to find a *first* feasible solution,
and it does that well.

The quality of the solution is the job of the improvement methods
of the next sections, which start from an incumbent.

Summary

- The **feasibility pump** alternates a rounding (an integer point) and a projection (the closest point of $\mathcal{L}$, one LP) until the two points coincide
- The distance $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ is the L_1 distance on the integer variables: linear for binary variables, with auxiliary variables d_j for general integers
- Short cycles are broken by a **flip**, long cycles by a random **restart**
- The pump ignores the objective: on $P72$ it finds a solution of value 2, against the optimal value 7

Improving an incumbent with a sub-MIP

Introduction: Improving an incumbent with a sub-MIP

- The methods of the previous sections build a solution from the LP relaxation
- The three methods of this section start from an **incumbent $\bar{x}$**, computed by any heuristic — rounding, diving, the feasibility pump — and try to improve it
- We see the common pattern (a sub-MIP with a cutoff constraint), then RINS, local branching and proximity search

The sub-MIP

- They all follow the same pattern: add to the MIP some constraints that restrict it to a set of solutions *related to* $\bar{\boldsymbol{x}}$, and solve the restricted problem, called the **sub-MIP**, with the MIP solver
- The sub-MIP is a neighbourhood of $\bar{\boldsymbol{x}}$, but a very large one, explored exactly by the solver rather than by enumeration
- It is the idea of **large neighbourhood search**, with the solver as the repair method

The cutoff constraint

- Only solutions *better* than the incumbent are of interest
- With an improvement step $\theta > 0$, the sub-MIP contains the **cutoff constraint**

$$f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta \quad \text{(minimization)},$$
$$f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}}) + \theta \quad \text{(maximization)}.$$

- When f takes integer values on all the feasible solutions, as for all the examples of this chapter, $\theta = 1$ excludes exactly the solutions that are not better than $\bar{\boldsymbol{x}}$

Local optimum, hard and soft fixing

- If the sub-MIP with the cutoff is infeasible, the neighbourhood contains no better solution: the solver has *proved* that $\bar{\boldsymbol{x}}$ is a local optimum for that neighbourhood
- The methods differ in *which* constraints define the neighbourhood:
  - RINS fixes variables (**hard fixing**)
  - local branching bounds the number of variables that change and proximity search minimizes it (**soft fixing**)

The first method compares the incumbent with the LP solution, and fixes the variables on which the two agree: **RINS**.

Definition 8: RINS

RINS (Relaxation Induced Neighbourhood Search; Danna, Rothberg and Le Pape, 2005) compares the incumbent $\bar{\boldsymbol{x}}$ with the solution $\tilde{\boldsymbol{x}}$ of the LP relaxation. The integer variables on which the two solutions agree are fixed to their common value; the sub-MIP on the remaining variables, with the cutoff constraint, is solved by the MIP solver.

- The pseudo-code of RINS is on the next two slides

Algorithm: RINS (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; improvement step θ
Output : A solution at least as good as $\bar{\boldsymbol{x}}$

```
1 function RINS(MIP, x̄, θ)
2 |   x̃ := optimal solution of the LP relaxation
3 |   for j ∈ 𝒟 do
4 |   |   if x̃_j = x̄_j then
5 |   |   |   fix x_j := x̄_j          // LP and incumbent agree
6 |   |   end
7 |   end
```

Algorithm: RINS (2/2)

function *RINS(MIP, $\bar{x}$, θ)* (continued)

```
8   |  add the cutoff constraint f(x) ≤ f(x̄) − θ
9   |  solve the sub-MIP with the MIP solver (with a node limit)
10  |  if the sub-MIP has a feasible solution x' then
11  |  |  return x'                          // better than x̄
12  |  else
13  |  |  return x̄          // no better solution in the
    |  |   neighbourhood
14  |  end
15  end
```

Why the agreement

- The LP solution is optimistic and possibly infeasible, the incumbent is feasible and possibly poor: two advisers with opposite defects
- Where they agree, the value is probably right in good solutions too
- Where they disagree, there is something to decide, and the decision is left to the solver

Size of the neighbourhood and hard fixing

- **The size of the neighbourhood is not chosen.** It is the number of variables on which $\tilde{\boldsymbol{x}}$ and $\bar{\boldsymbol{x}}$ disagree: small if the incumbent is already close to the LP solution, large otherwise. For this reason the sub-MIP is solved with a limit on the number of nodes

- **RINS inside branch-and-bound.** Danna, Rothberg and Le Pape apply RINS at the nodes of the branch-and-bound tree, with the LP solution of the node: deeper nodes have different LP solutions, hence different neighbourhoods of the same incumbent. Every modern MIP solver runs RINS in this way

- **Hard fixing.** A fixed variable cannot change in the sub-MIP. If a better solution needs to change a variable on which $\tilde{\boldsymbol{x}}$ and $\bar{\boldsymbol{x}}$ agree, RINS cannot find it

Example (1/2): RINS on *P7*

- The incumbent is $\bar{\boldsymbol{x}} = (2, 0)$, of value 6 (rounding down), and the LP solution is $\tilde{\boldsymbol{x}} = \left(\frac{5}{2}, 0\right)$. They agree on $x_2 = 0$, which is fixed. The sub-MIP is

$$\begin{aligned} \max \quad & 3\,x_1 \\ \text{subject to} \quad & x_1 \leq 12, \\ & 2\,x_1 \leq 5, \\ & 3\,x_1 \geq 7, \\ & x_1 \in \mathbb{Z}_{\geq 0} \end{aligned}$$

- It is infeasible: the second constraint gives $x_1 \leq 2$, the cutoff $x_1 \geq 3$. RINS returns the incumbent
- The optimal solution $(2, 1)$ has $x_2 = 1$, but x_2 is fixed to 0 because the LP and the incumbent agree on it

Example (2/2): RINS on $P42$

- The incumbent is $\bar{\boldsymbol{x}} = (2, 1, 0)$, of value 8 (rounding down), and $\tilde{\boldsymbol{x}} = \left(\frac{5}{2}, \frac{3}{2}, 0\right)$
- They agree on $x_3 = 0$ only
- The sub-MIP on x_1, x_2 with the cutoff $3x_1 + 2x_2 \geq 9$ is solved by Gurobi, which returns $(2, 2, 0)$, of value 10
- **RINS finds an optimal solution**

On $P7$ the variable that must change is exactly the one on which the LP and the incumbent agree;

on $P42$ the agreement concerns a variable that does not need to change, and the neighbourhood contains the optimum.

The local branching constraint

- RINS decides *which* variables may change. Local branching (Fischetti and Lodi, 2003) decides only *how many* of them may change, and leaves the choice of which ones to the solver
- For a MIP whose integer variables are binary (or, with the auxiliary variables d_j of the distance, general integers), the **local branching constraint** with radius $k \in \mathbb{Z}_{\geq 1}$ is

$$\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \;=\; \sum_{j \in \mathscr{D}:\ \bar{x}_j = 1} (1 - x_j) \;+\; \sum_{j \in \mathscr{D}:\ \bar{x}_j = 0} x_j \;\leq\; k$$

- $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ is the distance of the feasibility pump, for binary variables: on binary points, it counts the variables whose value differs from the incumbent
- The constraint is linear, so it can be added to the MIP like any other constraint

Definition 9: local branching

Local branching solves, with the MIP solver, the sub-MIP made of the MIP, the cutoff constraint and the local branching constraint $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$. *If it finds a better solution, the new solution becomes the incumbent and the neighbourhood is centred on it. If it proves that the neighbourhood contains no better solution, the radius* k *is enlarged. In both cases the explored neighbourhood is excluded from the following sub-MIPs by the reverse constraint* $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1$.

Algorithm: Local branching (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; radius k; maximum radius $k_{\max}$; step k_{step}; improvement step θ

Output : A solution at least as good as $\bar{\boldsymbol{x}}$

1 **function** *LocalBranching(MIP, $\bar{\boldsymbol{x}}$, k, $k_{\max}$, k_{step}, θ)*

2 $\quad \mathscr{R} := \emptyset$ `// reverse constraints added so far`

3 $\quad$ **while** $k \leq k_{\max}$ **do**

4 $\quad\quad$ solve the MIP with the constraints $\mathscr{R}$, $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ and $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$

5 $\quad\quad$ $\mathscr{R} \leftarrow \mathscr{R} \cup \left\{\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1\right\}$ `// never explore it again`

Algorithm: Local branching (2/2)

function *LocalBranching(MIP,* $\bar{\boldsymbol{x}}$*,* k*,* $k_{\max}$*,* k_{step}*,* θ*)*
(continued)

```
    while k ≤ k_max (continued) do
6       if a feasible solution x' is found then
7           x̄ ← x'        // re-centre the neighbourhood
8       else
9           k ← k + k_step  // no better solution within
              distance k
10      end
11  end
12  return x̄
13 end
```

Soft fixing

- With $|\mathscr{D}|$ binary variables, the neighbourhood $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ contains

$$\sum_{i=0}^{k} \binom{|\mathscr{D}|}{i}$$

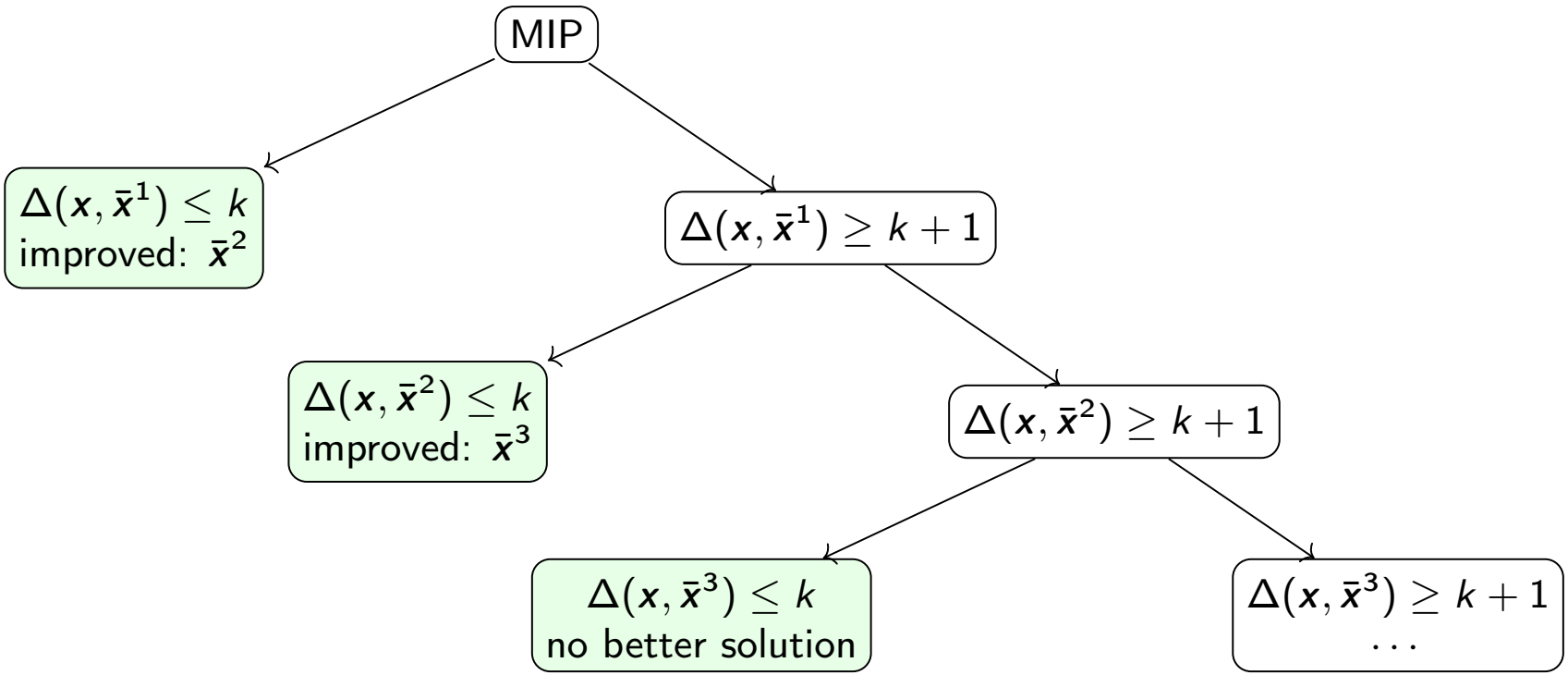
  binary points
- For k small it is much smaller than the whole space, but it does not commit to any particular variable

- Any k variables may change

A branching scheme

- The name comes from the original use of the constraint as a *branching rule*: the left branch $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ is solved first, and the right branch $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \geq k + 1$ contains the rest of the problem
- If the left branch improves the incumbent, the right branch is split again around the new incumbent
- The tree is a long chain of right branches (next slide)

The local branching tree



The radius and local search

- **The radius.** k must be small enough for the sub-MIP to be solved quickly, and large enough for the neighbourhood to contain better solutions
- The original method uses small values compared with $|\mathscr{D}|$, a time limit on each sub-MIP, and *diversification* — a larger k, or a jump to a different solution — when no improvement is found
- **Local branching and local search.** With $k = 1$ the neighbourhood contains the points obtained from $\bar{\boldsymbol{x}}$ by changing a single integer variable by one unit (a binary variable flips); with larger k it contains the points that differ from $\bar{\boldsymbol{x}}$ in several variables, or by several units
- It is a local search whose neighbourhood is explored exactly by the solver

Example (1/3): local branching on *P7*

- Start from $\bar{\boldsymbol{x}}^1 = (2, 0)$, of value 6, with $k = 1$, $k_{\max} = 3$, $k_{step} = 1$ and $\theta = 1$
- $k = 1$. The sub-MIP with $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \leq 1$ and the cutoff $3x_1 + x_2 \geq 7$ returns $(2, 1)$, of value 7: the new incumbent $\bar{\boldsymbol{x}}^2$, found at distance 1 (the variable x_2 changes by one unit). Add $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}^1) \geq 2$
- $k = 1, 2, 3$, **around** $\bar{\boldsymbol{x}}^2$. No solution of value at least 8: k becomes $4 > k_{\max}$ and the method stops with $(2, 1)$, the optimum
- Where RINS failed, local branching lets the solver choose that x_2 changes

Example (2/3): local branching on *P*72

- Start from the solution of the feasibility pump, $\bar{x}^1 = (0, 0, 1, 3, 1)$, of value 2, with the same parameters

k	around	solution returned	value
1	$\bar{x}^1$	$\bar{x}^2 = (0, 0, 1, 2, 1)$, at distance 1	4
1	$\bar{x}^2$	no solution of value ≥ 5	—
2	$\bar{x}^2$	$\bar{x}^3 = (0, 1, 1, 2, 0)$, at distance 2	7
2, 3	$\bar{x}^3$	no solution of value ≥ 8	—

Example (3/3): local branching on *P*72

- With $k_{\max} = 3$ the method stops with value 7, the optimum, after two improvements
- The radius enlarged from 1 to 2 because the improvement of x_2 and x_5 together needs two units of distance

Local branching performs a local search with a neighbourhood whose size is controlled by k, explored exactly by the solver, and it enlarges the neighbourhood when it gets stuck: exactly the idea of variable neighbourhood search, with the solver doing the exploration.

Local branching keeps the objective and constrains the distance.

Proximity search of Fischetti and Monaci (2014) does the opposite: it keeps the cutoff as a constraint and *minimizes* the distance.

Definition 10: proximity search

*The **proximity sub-MIP** around the incumbent $\bar{\boldsymbol{x}}$ reads*

$$
\begin{aligned}
\min \quad & \Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \\
\text{subject to} \quad & A\boldsymbol{x} \leq b, \\
& l_j \leq x_j \leq u_j, && \forall j \in \{1, 2, \ldots, N\}, \\
& f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta, \\
& x_j \in \mathbb{Z}, && \forall j \in \mathscr{D}.
\end{aligned}
$$

Proximity search solves it, replaces the incumbent with its solution, and repeats, until the proximity sub-MIP is infeasible.

Algorithm: Proximity search (1/2)

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; improvement step θ
Output : A solution at least as good as $\bar{\boldsymbol{x}}$

```
1 function ProximitySearch(MIP, x̄, θ)
2 |   while true do
3 |   |   solve min{Δ(x, x̄) : x feasible for the MIP, f(x) ≤
  |   |     f(x̄) − θ}
4 |   |   if the proximity sub-MIP is infeasible then
5 |   |   |   return x̄          // no solution better than
  |   |   |     f(x̄) − θ
6 |   |   end
```

Algorithm: Proximity search (2/2)

function *ProximitySearch(MIP, $\bar{\boldsymbol{x}}$, θ)* (continued)

    while *true* (continued) **do**

7 $\quad\quad\quad$ $\boldsymbol{x}' :=$ its optimal solution

8 $\quad\quad\quad$ fix $x_j := x'_j$ for $j \in \mathscr{D}$ and re-optimize f on the continuous variables

9 $\quad\quad\quad$ $\bar{\boldsymbol{x}} \leftarrow \boldsymbol{x}'$ `// the closest improving solution`

10 $\quad$ **end**

11 **end**

Why minimize the distance

- The original objective is often what makes a MIP hard: its LP relaxation drives the solver toward fractional points far from feasible solutions
- In the proximity sub-MIP the objective is the distance, and the LP solution stays close to $\bar{\boldsymbol{x}}$, which is almost feasible: the heuristics of the solver work well, and a feasible solution is usually found quickly
- In practice the proximity sub-MIP is not solved to optimality: any feasible solution is an improvement

No radius, continuous part, the step θ

- **No radius to choose.** The size of the move is decided by the sub-MIP itself: the smallest change that improves the objective by at least θ
- **Re-optimizing the continuous part.** The solution of the sub-MIP has the right integer part, but its continuous variables were chosen to minimize the distance, not f. Fixing the integer variables and solving the LP on the continuous ones gives the true value of the new incumbent
- **The step θ.** A large θ asks for large improvements, hence large moves and hard sub-MIPs; a small θ gives easy sub-MIPs and many iterations. With $\theta = 1$ and integer objective values, the method also proves optimality

Proposition 1

If f takes integer values on all the feasible solutions of the MIP and the proximity sub-MIP around $\bar{\boldsymbol{x}}$ with $\theta = 1$ is infeasible, then $f(\bar{\boldsymbol{x}})$ is the optimal value of the MIP.

Proof

- The feasible region of the proximity sub-MIP is the set of the feasible solutions $\boldsymbol{x}$ of the MIP with

$$f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - 1;$$

  it does not depend on the objective Δ
- If it is empty, every feasible solution has

$$f(\boldsymbol{x}) > f(\bar{\boldsymbol{x}}) - 1, \quad \text{hence} \quad f(\boldsymbol{x}) \geq f(\bar{\boldsymbol{x}}),$$

  since both values are integer
- So $\bar{\boldsymbol{x}}$ is optimal

□

Example (1/2): proximity search on $P7$

- From $\bar{\boldsymbol{x}} = (2, 0)$, of value 6: the closest solution of value at least 7 is $(2, 1)$, at distance 1
- Around it, no solution has value at least 8 (the LP bound is $\frac{15}{2}$)
- By the proposition above, $(2, 1)$ is **optimal**

Example (2/2): proximity search on *P*72

- From the solution of the feasibility pump, of value 2:

iteration	cutoff	min Δ	solution returned	value
1	$f(\boldsymbol{x}) \geq 3$	1	$(0, 0, 1, 2, 1)$	4
2	$f(\boldsymbol{x}) \geq 5$	2	$(0, 1, 1, 2, 0)$	7
3	$f(\boldsymbol{x}) \geq 8$	—	infeasible	—

- At the third iteration the sub-MIP is infeasible, and the proposition proves that 7 is optimal, although the upper bound $\lfloor z(\mathrm{LP}) \rfloor = 8$ of the root does not

The proof of optimality comes from the solver,
which proves the infeasibility of the last sub-MIP
with its own dual bounds:

proximity search, used with $\theta = 1$,
is an exact method that improves the incumbent
along the way.

Summary (1/2)

The common pattern:

- A **sub-MIP**: the MIP restricted to solutions related to the incumbent, explored exactly by the solver (large neighbourhood search)
- The **cutoff constraint** keeps only better solutions; an infeasible sub-MIP proves a local optimum

RINS:

- Fixes the variables on which the LP solution and the incumbent agree (**hard fixing**)
- Finds the optimum on $P42$, fails on $P7$, where the variable to change is a fixed one

Summary (2/2)

Local branching:

- Adds $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$ (**soft fixing**), re-centres on improvement, enlarges k when stuck
- Reaches the optimum on $P7$ and on $P72$

Proximity search:

- Minimizes $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ subject to the cutoff: no radius to choose
- With $\theta = 1$ and integer objective values, an infeasible proximity sub-MIP **proves optimality** ($P7$, $P72$)

Relax-and-fix and fix-and-optimize

Introduction: Relax-and-fix and fix-and-optimize

- Some MIPs are too large to be solved at once, but their variables come in blocks
- **Relax-and-fix** builds a solution block by block; **fix-and-optimize** improves it window by window
- We run both on $P46$, and fix-and-optimize also on $P72$

Blocks of variables

- The variables of many MIPs come in **blocks** with a natural order, for instance because they are indexed by a sequence, and the decisions of a block mainly interact with those of the nearby blocks
- The two methods of this section exploit a partition of the integer variables into blocks: they solve a sequence of small MIPs instead of a large one
- Once the integer variables are fixed, the rest is an LP: the hard decisions are the integer variables, and the LP relaxation is the tool that estimates the effect of the decisions not yet taken

Definition 11: relax-and-fix

Relax-and-fix splits the integer variables into blocks $B_1, B_2, \ldots, B_p$, in a given order. At step r it solves the MIP in which the variables of B_r are integer, those of $B_1, \ldots, B_{r-1}$ are fixed to the values decided at the previous steps, and those of $B_{r+1}, \ldots, B_p$ are relaxed (continuous). Then it fixes the variables of B_r to their values, and moves to the next block. The solution of the last step is feasible.

Algorithm: Relax-and-fix

Input : MIP; partition $B_1, B_2, \ldots, B_p$ of $\mathscr{D}$, in a given order
Output : Feasible solution, or FAIL

```
1  function RelaxAndFix(MIP, B_1, ..., B_p)
2      for r := 1 to p do
3          solve the MIP with: x_j fixed for j ∈ B_1 ∪ ··· ∪ B_{r-1}, x_j
             integer for j ∈ B_r, x_j continuous for
             j ∈ B_{r+1} ∪ ··· ∪ B_p
4          if it is infeasible then
5              return FAIL
6          end
7          x' := its optimal solution
8          fix x_j := x'_j for all j ∈ B_r
9      end
10     return x'          // all the integer variables are
         integer
11 end
```

Why it works, and its myopia

- **Why.** The decisions of the first blocks are taken with care (integer), while the others are only estimated (relaxed). Each MIP has the integer variables of a single block, so it is small
- **Myopia.** The relaxed blocks are optimistic: the relaxation can buy a fraction of a decision for a fraction of its cost. A decision that looks good because the rest is cheap may turn out to be bad when the rest becomes integer. The fixed decisions can even make a later step infeasible
- **Variants.** Larger blocks give better solutions and harder MIPs; *overlapping* blocks solve $B_r \cup B_{r+1}$ as integer but fix only B_r, to see a little further

Example (1/3): relax-and-fix on $P46$

- The integer variables are x_1, x_2, x_3
- The LP relaxation has value $\frac{499}{25}$, the upper bound is 19 and $z(\mathrm{ILP}) = 16$, attained by $(0, 1, 1)$
- **One variable per block,** $B_1 = \{x_1\}$, $B_2 = \{x_2\}$, $B_3 = \{x_3\}$:

step r	value of the MIP of step r	solution of the MIP	fixing
1	$\frac{296}{15} \approx 19.73$	$\left(0, \frac{26}{15}, \frac{14}{15}\right)$	$x_1 := 0$
2	$\frac{56}{3} \approx 18.67$	$\left(0, 2, \frac{2}{3}\right)$	$x_2 := 2$
3	12	$(0, 2, 0)$	$x_3 := 0$

Example (2/3): relax-and-fix on $P46$

- The result is $(0, 2, 0)$, of value 12, with a gap of

$$\frac{19 - 12}{12} \approx 58.3\%$$

- The wrong decision is taken at step 2: with x_3 relaxed, $x_2 = 2$ gives $\frac{56}{3} \approx 18.67$ and $x_2 = 1$ gives $\frac{122}{7} \approx 17.43$, so $x_2 := 2$

- But then the second constraint leaves room only for $x_3 \leq \frac{2}{3}$, and x_3 becomes 0 when it is integer

Example (3/3): relax-and-fix on $P46$

- **Larger blocks,** $B_1 = \{x_1\}$, $B_2 = \{x_2, x_3\}$. Step 1 is the same
- At step 2 the MIP on x_2, x_3 is solved with both variables integer: it returns $(0, 1, 1)$, of value 16, the **optimum**
- The size of the blocks trades the quality of the solution against the difficulty of the MIPs: the larger block sees the interaction between x_2 and x_3 that the single-variable blocks miss

Definition 12: fix-and-optimize

Fix-and-optimize improves an incumbent $\bar{\boldsymbol{x}}$. It considers a sequence of windows $W_1, W_2, \ldots, W_q \subseteq \mathscr{D}$, typically consecutive periods. For each window it fixes all the integer variables outside the window to their values in $\bar{\boldsymbol{x}}$, and solves the sub-MIP on the variables of the window, with the cutoff constraint. A better solution becomes the incumbent. The passes over the windows are repeated until a whole pass gives no improvement.

Algorithm: Fix-and-optimize

Input : MIP; incumbent $\bar{\boldsymbol{x}}$; windows $W_1, \ldots, W_q$; improvement step θ

Output : A solution at least as good as $\bar{\boldsymbol{x}}$

```
1  function FixAndOptimize(MIP, x̄, W_1, ..., W_q, θ)
2      improved := true
3      while improved do
4          improved ← false
5          for r := 1 to q do
6              solve the MIP with x_j := x̄_j for j ∈ 𝒟 \ W_r and
                 f(x) ≤ f(x̄) − θ
7              if a feasible solution x' is found then
8                  x̄ ← x';    improved ← true
9              end
10         end
11     end
12     return x̄
13 end
```

Example (1/3): fix-and-optimize on $P46$

- Start from the solution of relax-and-fix, $\bar{\boldsymbol{x}} = (0, 2, 0)$, of value 12, with the windows $W_1 = \{x_1\}$ and $W_2 = \{x_2, x_3\}$ and $\theta = 1$

window	incumbent after the window	
$\{x_1\}$	12	no improvement ($x_1 = 1$ violates the first constraint)
$\{x_2, x_3\}$	16	$\bar{\boldsymbol{x}} = (0, 1, 1)$
second pass, both windows	16	no improvement: stop

Example (2/3): fix-and-optimize on $P46$

- With the three windows of one variable each, $\{x_1\}$, $\{x_2\}$, $\{x_3\}$, no window improves $(0, 2, 0)$: it is a local optimum for those windows
- The window $\{x_2, x_3\}$ revises the wrong decision of relax-and-fix and reaches the optimum 16

Example (3/3): fix-and-optimize on *P*72

- Start from the solution of the feasibility pump, $(0, 0, 1, 3, 1)$, of value 2, with the windows $\{x_1, x_2\}$, $\{x_3, x_4\}$, $\{x_5\}$
- The first window gives no improvement; the second gives $(0, 0, 2, 4, 1)$, of value 5; the third and the second pass give none
- The method stops with $5 < 7 = z(\mathrm{ILP})$: a local optimum with respect to these windows
- The optimal solution $(0, 1, 1, 2, 0)$ differs from it in x_2, x_3, x_4 and x_5, more variables than any window contains

Local search with windows

- Fix-and-optimize is a **local search** whose moves change all the integer variables of a window at once, explored exactly by the solver: the integer-programming counterpart of the neighbourhoods of local search
- It stops in a local optimum with respect to all the windows
- Relax-and-fix and fix-and-optimize are natural partners: the first builds a solution block by block, the second repairs the myopic decisions looking at several blocks together

Summary

- **Relax-and-fix**: one block integer, the previous ones fixed, the next ones relaxed; then fix the block and move on
- It is **myopic**: on $P46$ single-variable blocks give 12, the block $\{x_2, x_3\}$ gives the optimum 16
- **Fix-and-optimize**: re-optimize the variables of a window, the rest fixed to the incumbent, with the cutoff; repeat the passes until no improvement
- It stops in a local optimum for the windows: 16 on $P46$, $5 < 7$ on $P72$

Heuristics inside the solver

Introduction: Heuristics inside the solver

- A modern MIP solver is not only a branch-and-bound algorithm: it runs, automatically, most of the heuristics of this chapter
- We see when Gurobi runs them, and the parameters that control the effort spent in them
- We also see how to pass to the solver a solution computed outside it

When the heuristics run

- The chapter «MIP solvers: what they do» shows real logs. In Gurobi:
  - *before the root LP*, heuristics that need no relaxation look for a first solution
  - *at the root*, after the LP, rounding, several diving heuristics and the feasibility pump are run
  - *during the branch-and-bound*, RINS and other sub-MIP heuristics are run at some nodes, with the LP solution of the node
- The new incumbents are reported in the log by a line starting with `H` (heuristic) instead of $*$ (integer LP solution of a node)

The parameters of the heuristics

- The effort spent in heuristics is controlled by parameters. The most important ones are:

parameter	default	meaning
`Heuristics`	0.05	fraction of the work spent in the heuristics (0 to 1)
`MIPFocus`	0	1: focus on finding good feasible solutions rather than on the bound
`RINS`	-1	frequency of RINS (every n nodes; -1 automatic, 0 off)
`PumpPasses`	-1	passes of the feasibility pump at the root (-1 automatic)
`SubMIPNodes`	500	node limit of the sub-MIPs of RINS and similar heuristics
`NoRelHeurWork`	0	work spent in the heuristic without relaxation before the root
`ImproveStartGap`	0	when the gap reaches this value, switch to improving the incumbent only

MIP start

- A heuristic solution computed outside the solver — by any of the methods of this chapter — can be passed to it as a **MIP start** (the attribute `Start` of the variables)
- The solver starts from it as incumbent, uses it for pruning and for RINS, and never returns anything worse
- The numbers depend on the instance, on the version of the solver and on the machine; a single instance proves nothing
- The lesson is qualitative: when the bound is strong, the time to solve is the time to find a good solution, and the heuristics of the solver — or a good MIP start — decide it

Summary

- Gurobi runs heuristics before the root LP, at the root (rounding, diving, feasibility pump) and during the branch-and-bound (RINS and other sub-MIP heuristics); `H` lines in the log
- Parameters such as `Heuristics`, `MIPFocus`, `RINS`, `PumpPasses`, `SubMIPNodes` control the effort
- An external heuristic solution enters as a **MIP start**

What MIP-based heuristics give

Introduction: What MIP-based heuristics give

- A heuristic solution alone says nothing about its quality
- A MIP-based heuristic comes with a dual bound: we compute the gap of all the solutions found in the chapter
- We close with a table that compares all the methods

A heuristic that knows how far it is

- A heuristic gives only one side of the gap, and without a dual bound nothing can be said about the quality of its solution
- A MIP-based heuristic works with a MIP model, and the model comes with its LP relaxation; run inside a MIP solver, it comes with the dual bound of the branch-and-bound as well
- The heuristic **knows how far it is from the optimum** while it runs: the optimality gap

$$\text{optimality gap} = \frac{\text{UB} - \text{LB}}{|\text{LB}|}$$

is computed at every moment, and when it is 0 the solution is *proved* optimal

The gaps of the examples

instance	solution	LB	UB $= \lfloor z(\mathrm{LP}) \rfloor$	gap	certified by
car, rounding down	$(3, 2)$	13	15	15.4%	—
car, diving	$(4, 1)$	14	15	7.1%	proximity search from $(3, 2)$
*P*7, rounding down	$(2, 0)$	6	7	16.7%	—
*P*7, local branching	$(2, 1)$	7	7	0%	the bound
*P*42, RINS	$(2, 2, 0)$	10	10	0%	the bound
*P*46, relax-and-fix	$(0, 2, 0)$	12	19	58.3%	—
*P*46, fix-and-optimize	$(0, 1, 1)$	16	19	18.8%	proximity search
*P*72, feasibility pump	$(0, 0, 1, 3, 1)$	2	8	300%	—
*P*72, local branching	$(0, 1, 1, 2, 0)$	7	8	14.3%	proximity search

Finding or proving

- On *P*7 and *P*42 the integer rounding of the LP value is already the optimal value: the difficulty is to find a solution that attains the bound, not to prove it
- On the other instances the bound $\lfloor z(\mathrm{LP}) \rfloor$ is not tight, and the certificate comes from proximity search (the optimality proposition) or from the branch-and-bound of the solver

A MIP-based heuristic does not only find solutions:
inside a MIP solver it works with a dual bound,
so the gap of its solution is always known,
and a gap equal to 0 certifies optimality.

The methods of this chapter (1/2)

method	needs LP	needs incumbent	fixing	idea
rounding	yes	no	—	round the LP solution
diving	yes	guided: yes	one at a time	fix a variable, re-solve the LP, repeat
feasibility pump	yes	no	—	alternate rounding and projection until they meet
RINS	yes	yes	hard	fix where LP and incumbent agree, solve the sub-MIP

The methods of this chapter (2/2)

method	needs LP	needs incumbent	fixing	idea
local branching	no	yes	soft	sub-MIP with $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}}) \leq k$
proximity search	no	yes	soft	minimize $\Delta(\boldsymbol{x}, \bar{\boldsymbol{x}})$ subject to the cutoff
relax-and-fix	yes	no	by blocks	integer now, relaxed later, then fix
fix-and-optimize	no	yes	by windows	re-optimize a window, the rest fixed

Summary

- With a dual bound, the **optimality gap** $\frac{\text{UB}-\text{LB}}{|\text{LB}|}$ of a heuristic solution is known at every moment
- On the examples the gaps go from 300% (feasibility pump on $P72$) to 0% (local branching on $P7$, RINS on $P42$)
- When the bound is not tight, the certificate comes from proximity search or from the branch-and-bound
- Methods differ in what they need (LP, incumbent) and in how they fix variables (none, one at a time, hard, soft, by blocks, by windows)

Python implementation

Introduction: `Python` implementation

- All the methods of the chapter are implemented in Python with `gurobipy`
- We see how the codes are organized, and two short functions: rounding and proximity search
- The codes reproduce every example of the chapter

The organization of the codes

- The codes use Python 3 and `gurobipy`. The variables of a model are accessed by their position j in `m.getVars()`, and the list `I` of the positions of the integer variables is the set $\mathscr{D}$; indices start from 0
- The module `common.py` contains the environment of the solver and the operations used by all the methods: the LP relaxation, the rounding $[\cdot]$, the completion of a point, the distance Δ and the cutoff constraint
- The model of the examples (`models.py`): `max c x` subject to `A x <= b`, with integer variables in $\mathbb{Z}_{\geq 0}$ and optional upper bounds

Rounding in Python

```
1 from common import *
2
3 def rounding(m, trace=None):
4     I = int_index(m)
5     x_star = solve_lp(m)
6     note(trace, "LP", x_star)
7     x_tilde = round_point(x_star, I)
8     note(trace, "rounded", x_tilde)
9     return evaluate(m, x_tilde)
```

- `solve_lp`: the optimal solution of the LP relaxation
- `round_point`: $\bar{x}_j = [\tilde{x}_j]$ for every integer variable
- `evaluate` is the completion: `(value, x)` if feasible, `None` otherwise

Proximity search in Python

```
def proximity_search(m, x_tilde, theta=1, trace=None):
    I = int_index(m)
    z_tilde = obj_of(m, x_tilde)
    while True:
        p = m.copy()
        add_cutoff(p, z_tilde, theta)
        p.setObjective(distance(p, x_tilde, I),
                       GRB.MINIMIZE)
        res = solve_mip(p)
        if res is None:
            note(trace, "infeasible")
            return z_tilde, x_tilde
        z_new, x_new = evaluate(m, res[1])
        note(trace, "found", res[0], z_new, x_new)
        z_tilde, x_tilde = z_new, x_new
```

The files of the codes

- `common.py`, `models.py`: the shared operations and the model of the examples
- `rounding.py`, `diving.py`, `feasibility_pump.py`
- `rins.py`, `local_branching.py`, `proximity_search.py`
- `relax_and_fix.py`, `fix_and_optimize.py`
- All the functions are in the folder `MIP_HEUR_CODE`, together with `instances.py`, which contains the instances of the chapter and of the exercises
- The command `python3 run_tests.py` runs every example of the chapter and every instance of the exercises, checking each value against the expected value, written in the file

Summary

- One Python function per method, built on the shared operations of `common.py`: LP relaxation, rounding, completion, distance, cutoff
- The sub-MIPs are copies of the model with fixed bounds or extra constraints, solved by Gurobi
- `run_tests.py` checks every example of the chapter and every instance of the exercises

Typical mistakes

Introduction: Typical mistakes

- We list the mistakes that are made most often with MIP-based heuristics
- They concern what each method can and cannot do: rounding, diving, the pump, hard fixing, the cutoff, the proof of optimality and the myopia of relax-and-fix
- Knowing them helps to choose the right method and to read the results correctly

Typical mistakes

Observation 2: Typical mistakes (1/3)

- ***Rounding a problem with equality constraints.*** *Independent roundings of the variables of an equality constraint almost never satisfy it: rounding fails, and a dive is needed.*
- ***Believing that a dive can undo a decision.*** *A dive is one path of the tree, without backtracking: a wrong fixing is never revised, and a fixing that makes the LP infeasible ends the dive.*
- ***Using the feasibility pump to get a good solution.*** *After the first LP the pump ignores the objective: on P72 it returns a solution of value 2, against the optimal value 7.*

Typical mistakes

Observation: Typical mistakes (2/3)

- ***Trusting hard fixing.*** *RINS fixes the variables on which the LP and the incumbent agree, so it cannot find a better solution that changes one of them (P7).*
- ***Forgetting the cutoff constraint.*** *Without* $f(\boldsymbol{x}) \leq f(\bar{\boldsymbol{x}}) - \theta$ *the sub-MIP returns the incumbent solution again; with it, an infeasible sub-MIP proves a local optimum.*
- ***Claiming optimality without the conditions.*** *The optimality proposition of proximity search needs* $\theta = 1$ *and integer objective values on all the feasible solutions.*

Typical mistakes

Observation: Typical mistakes (3/3)

- ***Ignoring the myopia of relax-and-fix.*** *The relaxed blocks are optimistic: a decision that looks cheap can force a bad one later (P46).*
- ***Generalizing from one instance or one solver run.*** *Paths and times depend on the version of the solver and on the tie-breaking rules, so the lessons are qualitative.*

Summary

Typical mistakes:

- Rounding fails with equality constraints; a dive never revises a decision
- The feasibility pump finds feasibility, not quality
- Hard fixing can exclude the better solutions; the cutoff constraint is essential
- Optimality from proximity search needs $\theta = 1$ and integer objective values
- Relax-and-fix is myopic; one instance or one run proves nothing

In summary

Introduction: In summary

- We list the references of the chapter
- We collect the key ideas of the chapter
- We close with the dashboard of the car example: the bounds found so far

References

- T. Achterberg, T. Berthold, Improving the feasibility pump, *Discrete Optimization* 4 (2007) 77–86
- L. Bertacco, M. Fischetti, A. Lodi, A feasibility pump heuristic for general mixed-integer problems, *Discrete Optimization* 4 (2007) 63–76
- E. Danna, E. Rothberg, C. Le Pape, Exploring relaxation induced neighborhoods to improve MIP solutions, *Mathematical Programming* 102 (2005) 71–90
- M. Fischetti, F. Glover, A. Lodi, The feasibility pump, *Mathematical Programming* 104 (2005) 91–104
- M. Fischetti, A. Lodi, Local branching, *Mathematical Programming* 98 (2003) 23–47
- M. Fischetti, M. Monaci, Proximity search for 0-1 mixed-integer convex programming, *Journal of Heuristics* 20 (2014) 709–731

Summary (1/2)

- MIP-based heuristics use the LP relaxation to decide where to look and a MIP solver to explore a large neighbourhood exactly
- Rounding and diving build a solution from the LP; the feasibility pump looks only for feasibility; RINS, local branching and proximity search improve an incumbent solution with a sub-MIP
- Relax-and-fix builds a solution block by block, fix-and-optimize improves it window by window
- Inside a MIP solver the dual bound is available: the gap of the heuristic solution is known, and a gap equal to 0 certifies optimality

Summary (2/2)

Dashboard of the car example. The lower and upper bounds on $z(\mathrm{ILP})$ found so far in this module:

Chapter	Lower bound	Upper bound	From
Integer linear programming	12, 14	–	heuristic solutions $(4, 0)$ and $(4, 1)$
LP relaxation and dual bounds	–	15	$\lfloor z(\mathrm{LP}) \rfloor$, with $z(\mathrm{LP}) = \frac{31}{2}$
Cutting plane algorithm	14	14	5 Gomory fractional cuts
LP-based branch-and-bound	14	14	6 LP problems solved
This chapter, rounding down	13	15	$(3, 2)$
This chapter, diving	14	15	$(4, 1)$, certified optimal by proximity search